

Procurement Audit-Ready Assessment & Knowledge Harnessing

An officer-first procurement evaluation platform that makes tender decisions transparent, consistent, and defensible.

Reference Artifact Montage

Real sample files from the project workspace used as tender-side, bidder-side, and audit-side proof objects

Core principle: The system does not make sovereign eligibility decisions. It assembles evidence, applies deterministic rules, and preserves a reviewable trail so procurement officers can explain every outcome.

Prepared on: 6 May 2026

Table of Contents

Placeholder for table of contents	0
-----------------------------------	---

1. Executive Summary

PARAKH is a prototype for AI-assisted tender evaluation in Indian government procurement, designed specifically around the realities of CRPF-style eligibility analysis. Its defining principle is that AI should not become a black-box sovereign decision-maker. Instead, the platform should help officers work faster while preserving explainability, consistency, and a defensible audit trail.

In practice, PARAKH connects tender-side rules, bidder-side evidence, deterministic evaluation logic, and officer-led overrides into one coherent workflow. This manual serves two audiences at once: technical users who need to install and run the platform, and operational users or presenters who need to understand what the platform does, when to use each feature, and how to explain its outputs.

- Government tender evaluation is document-heavy, time-sensitive, and vulnerable to inconsistent interpretation.
- Bid packs contain mixed formats such as digital PDFs, scans, photos, handwritten notes, annexures, and corrigenda.
- Officers face personal accountability under RTI, vigilance review, audit, and legal challenge if a decision cannot be defended.
- Generic black-box AI is risky because OCR uncertainty, ambiguous clauses, and subjective criteria can cause silent errors.

2. Platform Overview

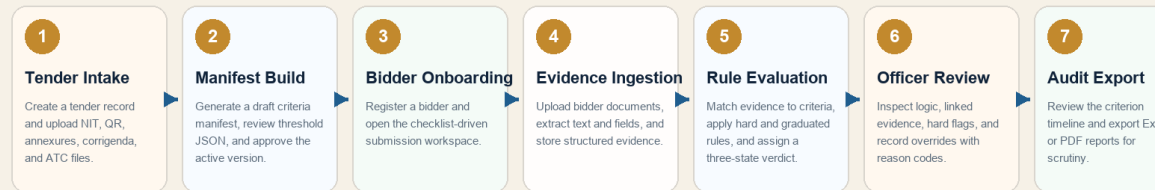
An officer-first procurement evaluation platform that makes tender decisions transparent, consistent, and defensible.

The platform is organized into four tightly connected subsystems.

Module	Role in the platform
TUE - Tender Understanding Engine	Creates a versioned criteria manifest from tender-side rules, staged into administrative, technical, and capacity checks.
BDI - Bidder Document Intelligence	Accepts bidder uploads, extracts text and fields, and turns heterogeneous documents into structured evidence.
EARE - Evaluation and Ambiguity-Resolution Engine	Maps evidence to criteria and produces deterministic three-state verdicts: Eligible, Not Eligible, or Needs Manual Review.
HLWAS - Human-in-the-Loop Workbench and Audit System	Lets officers review criterion-level logic, record overrides with reason codes, inspect audit history, and export reports.

Solution Workflow

Operational view of how tender rules and bidder evidence move through PARAKH from intake to export.



How the workflow maps to the codebase

- Tender Intake and Manifest Build map to the TUE flow in the tender router and tender service.
- Bidder Onboarding and Evidence Ingestion map to the BDI flow in the bidder router and bidder service.
- Rule Evaluation and Officer Review map to the EARE and workbench flows in the evaluation service and evaluation page.
- Audit Export maps to the audit router and audit service, which generate xlsx and pdf outputs from the append-only event history.

Why the solution fits the theme and problem

- PARAKH matches the theme by applying AI only where it adds traceable operational value: document understanding, structured extraction, and decision support.
- Deterministic rules handle objective criteria, while uncertain or subjective cases are escalated to officers instead of being auto-rejected.
- The solution is tailored to procurement realities such as tender-side versioning, bidder-side evidence capture, and audit-ready export.
- The platform directly addresses the explanation gap by preserving evidence, reasoning, overrides, and tamper-evident event history.

3. System Prerequisites and Installation

The repository supports three practical operating tracks: Docker Compose for integrated startup, a Windows quick-start path using included batch scripts and local runtime assets, and manual local development for backend and frontend work.

3.1 Recommended software baseline

Component	Recommended version or expectation	Purpose
Node.js	20.x or compatible runtime	Run the Next.js frontend
Python	3.11 or compatible	Run FastAPI backend and document-processing libraries
MongoDB	7.x or 8.x	Primary operational database
Redis	7.x (optional in current prototype)	Reserved for future async worker integration

Component	Recommended version or expectation	Purpose
Tesseract OCR	Installed locally for image-based extraction	Improve OCR on scanned bidder files
Docker Desktop	Current version	Run the complete stack through docker-compose

3.2 Docker setup

- 1 Open a terminal in the parakh/ folder.
- 2 Run `docker-compose up --build`.
- 3 Wait for the frontend, backend, MongoDB, and Redis containers to become healthy.
- 4 Open `http://localhost:3000` for the frontend and `http://localhost:8000/docs` for backend API documentation.

3.3 Local Windows quick start using included scripts

- 1 Open three separate terminals from the parakh/ root.
- 2 Run `.\start-mongo-local.cmd` to start the embedded MongoDB server against `local-runtime/data/db`.
- 3 Run `.\start-backend-local.cmd` to start the FastAPI backend on port 8000.
- 4 Run `.\start-frontend-local.cmd` to start the Next.js frontend on port 3000.

The local runtime folder already includes a MongoDB distribution and downloaded Tesseract installer assets for Windows-oriented demonstration flows.

3.4 Manual backend setup

- 1 Change into `parakh/backend`.
- 2 Create a virtual environment: `python -m venv .venv`
- 3 Activate the environment.
- 4 Install dependencies: `pip install -r requirements.txt`
- 5 Start the API: `uvicorn app.main:app --reload`

3.5 Manual frontend setup

- 1 Change into `parakh/frontend`.
- 2 Install frontend packages: `npm install`
- 3 Set `NEXT_PUBLIC_API_BASE` to `http://localhost:8000/api` if required.
- 4 Run `npm run dev`.
- 5 On Windows PowerShell, use `cmd /c npm.cmd install` and `cmd /c npm.cmd run dev` if the `npm shim` is blocked by script policy.

3.6 Environment variables and configuration

Variable	Purpose
MONGO_URI	MongoDB connection string used by the backend.
MONGO_DB	Target database name; default is parakh.
REDIS_URL	Reserved queue/cache connection for future asynchronous processing.
JWT_SECRET	Secret used to sign and verify access tokens.
JWT_ALGORITHM	JWT signing algorithm; the prototype defaults to HS256.
NEXT_PUBLIC_API_BASE	Frontend base URL for the backend API.
TESSERACT_CMD	Optional explicit path to the Tesseract executable.
TESSDATA_PREFIX	Optional tessdata directory override for OCR language packs.

4. Getting Started: Run the Platform End-to-End

- 1 Start MongoDB first, then backend, then frontend.
- 2 Open the frontend in the browser and let the auto-login flow acquire the seeded officer token.
- 3 Confirm backend health at /health and frontend data loading on the dashboard.
- 4 Open the sample CRPF tender or create a new tender.
- 5 Upload tender-side documents and approve the manifest.
- 6 Register a bidder, upload checklist documents, ingest evidence, evaluate the bidder, and export reports.

Hackathon Demo Flow

A judge-friendly script for demonstrating the platform in under five minutes.

01	Open the dashboard and show seeded activity counts. Use the Dashboard, Tenders, and Bidder Hub navigation.
02	Open the CRPF sample tender and upload supporting tender files if needed. Point to tender metadata, status, and tender document history.
03	Generate or review the manifest to show structured criteria and thresholds. Show stage, factor, thresholds, and approval status.
04	Register a bidder and upload checklist documents from the sample bidder folder. Explain that checklist status is updated document by document.
05	Run document ingestion and point out extracted evidence fields. Call out extraction method, text preview, and confidence-aware field capture.
06	Evaluate the bidder and walk through Eligible, Not Eligible, and Manual Review logic. Open the workbench and trace evidence to a specific criterion.
07	Record an officer override, open the audit trail, and export Excel or PDF. End on the audit timeline to reinforce trust and defensibility.

Seeded account	Role	Use case
admin	ADMIN	Administrative setup

Seeded account	Role	Use case
officer	OFFICER	Day-to-day review and evaluation

5. User Roles, Access, and Navigation

The backend exposes JWT-protected APIs and supports two roles: ADMIN and OFFICER. The frontend currently auto-signs in with the seeded officer account for demonstration convenience, but the backend still enforces role checks on protected actions.

Role	Typical responsibilities
ADMIN	Provision or supervise tenders, review manifests, and support operating governance.
OFFICER	Upload documents, evaluate bidders, inspect evidence, record overrides, and export audit reports.

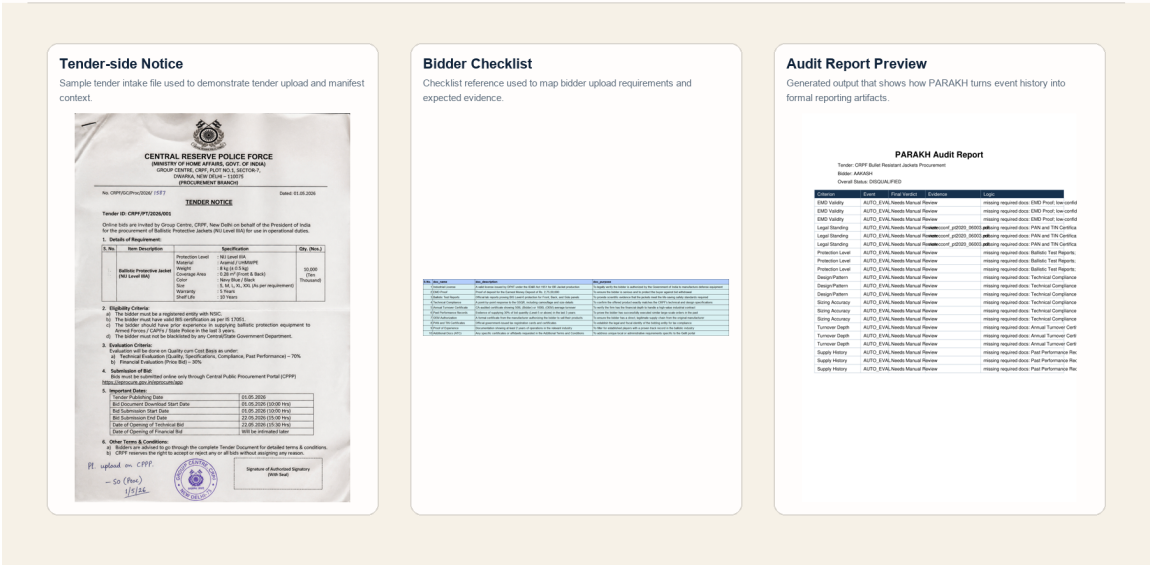
Primary navigation surfaces are Dashboard, Tenders, Bidder Hub, the Tender Detail page, the Bidder Console, the Evaluation Workbench, and the Audit Timeline.

6. Feature Documentation

This section documents every core platform function in an operator-first format.

Reference Artifact Montage

Real sample files from the project workspace used as tender-side, bidder-side, and audit-side proof objects.



6.1 Dashboard and Tender Portfolio

Provide a high-level operating picture by combining tender volume, bidder activity, disqualification count, override count, and audit-event count in one officer-first landing page.

Attribute	Detail
Purpose	Provide a high-level operating picture by combining tender volume, bidder activity, disqualification count, override count, and audit-event count in one officer-first landing page.

Attribute	Detail
Prerequisites	Backend API running Frontend running Officer login token available (the UI auto-signs in with the seeded officer account)
Inputs	Authenticated session Existing tender and bidder data in MongoDB
Outputs	Dashboard metrics Links into the tender portfolio and bidder workflow

Procedure

- 1 Open the frontend root page and confirm the officer profile pill is visible.
- 2 Review the metric tiles for Tenders, Bidders, Disqualifications, Auto Resolved, Overrides, and Audit Events.
- 3 Open an active tender from the dashboard or navigate to the Tenders page to continue the workflow.

Operational Notes

- The dashboard statistics are generated from the audit service and MongoDB counts, so they reflect current platform state.

6.2 Tender Creation

Create the authority-side container that will hold procurement metadata, uploaded tender documents, and the manifest approval trail.

Attribute	Detail
Purpose	Create the authority-side container that will hold procurement metadata, uploaded tender documents, and the manifest approval trail.
Prerequisites	Officer or admin role Frontend access to /tenders Backend API /api/tenders available
Inputs	Tender title Procuring entity Bid reference Optional description Optional bid validity end date
Outputs	A new tender record with status ACTIVE and a dedicated document workspace

Procedure

- 1 Navigate to the Tenders page and complete the Create Tender form.
- 2 Submit the form to create the tender record in MongoDB.
- 3 Confirm the new tender appears in the Tender Portfolio list and open Manage Manifest or Open Bidders to continue.

Operational Notes

- The repo seeds a sample CRPF tender automatically at startup, so tender creation can be demonstrated either from scratch or using seeded data.

6.3 Tender Document Upload

Attach the source tender-side files that define procurement rules, attachments, corrigenda, and terms.

Attribute	Detail
Purpose	Attach the source tender-side files that define procurement rules, attachments, corrigenda, and terms.
Prerequisites	Tender already created Officer or admin role
Inputs	Document type such as NIT, QR, Annexure, Corrigendum, or ATC Selected file upload
Outputs	Stored tender document under /uploads/tenders// Updated tender document history

Procedure

- 1 Open the Tender Detail page for the target tender.
- 2 Choose the document type from the dropdown.
- 3 Select the tender-side file and click Upload Tender Document.
- 4 Verify that the file appears in the Tender Documents list with filename, type, and upload timestamp.

Operational Notes

- Document uploads are traceable by document id and relative path and become the evidence base for manifest generation.

6.4 Manifest Generation, Review, and Approval

Produce the structured criteria blueprint used for evaluation and ensure only approved manifest versions can drive verdicting.

Attribute	Detail
Purpose	Produce the structured criteria blueprint used for evaluation and ensure only approved manifest versions can drive verdicting.
Prerequisites	Tender exists Officer or admin role Tender document workspace opened
Inputs	Generate Draft action Optional edits to criterion description, threshold JSON, or guidance notes

Attribute	Detail
Outputs	Draft or approved criteria manifest version latest_manifest_id linkage on the tender record

Procedure

- 1 Click Generate Draft on the Tender Detail page to create a new manifest version from the seeded evaluation criteria.
- 2 Review each criterion by stage, factor, rule type, classification, threshold JSON, and guidance notes.
- 3 Adjust descriptions or thresholds if the tender requires localized interpretation.
- 4 Click Save Draft to persist edits.
- 5 Click Approve Manifest when the version is ready for bidder evaluation.

Operational Notes

- Only DRAFT manifests are editable. Approving a new manifest supersedes any earlier APPROVED version for the same tender.

6.5 Bidder Registration

Create a bidder-side record scoped to the tender and initialize the checklist that governs required uploads.

Attribute	Detail
Purpose	Create a bidder-side record scoped to the tender and initialize the checklist that governs required uploads.
Prerequisites	Target tender exists Officer or admin role
Inputs	Bidder name Bidder type (OEM or Authorized Bidder) Optional OEM name
Outputs	Bidder record with PENDING overall status Checklist status initialized from fixture CSV

Procedure

- 1 Open the tender's Bidder Console.
- 2 Enter bidder identity details in the Register Bidder form.
- 3 Submit the form and confirm the bidder appears in the left-side bidder list.
- 4 Select the bidder to display the document checklist and subsequent actions.

Operational Notes

- The bidder type matters because non-OEM bidders require OEM Authorization evidence to avoid administrative disqualification.

6.6 Bidder Document Upload

Collect every bidder-side file expected by the checklist and maintain per-document status visibility.

Attribute	Detail
Purpose	Collect every bidder-side file expected by the checklist and maintain per-document status visibility.
Prerequisites	Bidder already registered Checklist visible on the Bidder Console
Inputs	Checklist document name Uploaded file for that document slot
Outputs	Stored bidder document under /uploads/bidders/// Checklist item marked Uploaded

Procedure

- 1 Choose a file for a checklist row such as Industrial License, EMD Proof, or Technical Compliance.
- 2 Click Upload for the relevant row.
- 3 Repeat until all required documents are uploaded or until the demo scope is satisfied.
- 4 Use the uploaded filename and status pill to confirm each checklist state update.

Operational Notes

- The checklist is driven by backend/app/fixtures/bidder_doc_checklist.csv, so it can be adapted tender by tender in future releases.

6.7 Evidence Ingestion and Extraction

Transform bidder submissions into text previews, structured fields, extraction methods, and confidence scores suitable for evaluation.

Attribute	Detail
Purpose	Transform bidder submissions into text previews, structured fields, extraction methods, and confidence scores suitable for evaluation.
Prerequisites	At least one bidder document uploaded OCR/runtime dependencies available for image-based files
Inputs	Ingest Documents action
Outputs	Extracted evidence records in MongoDB Field-level confidence scores and text preview

Procedure

- 1 On the selected bidder's checklist panel, click Ingest Documents.
- 2 Allow the backend to process each file according to its type: digital PDF, image, HTML, or fallback.
- 3 Open the Evaluation Workbench after ingestion and inspect the linked evidence to confirm extraction results.

Operational Notes

- Digital-text PDFs are read through PyMuPDF or pdfplumber first. OCR is only used when native text extraction is not available.
- The prototype currently extracts fields such as PAN, GSTIN, EMD amount/date, ballistic levels, turnover averages, size mix, OEM authorization, and experience years.

6.8 Evaluation Workbench

Give officers criterion-level visibility into the rules applied, evidence used, extracted values, and automatic verdict generated.

Attribute	Detail
Purpose	Give officers criterion-level visibility into the rules applied, evidence used, extracted values, and automatic verdict generated.
Prerequisites	Approved manifest available Bidder evidence already ingested
Inputs	Evaluate Bidder action or Re-run Evaluation action
Outputs	New AUTO_EVAL events Updated bidder overall status Disqualification reasons when applicable

Procedure

- 1 Click Evaluate Bidder from the Bidder Console or Re-run Evaluation from the workbench.
- 2 Select a criterion from the left-side event list.
- 3 Review stage, criterion name, auto verdict, final verdict, evidence quality, and event hash.
- 4 Inspect text previews and extracted values associated with the criterion.

Operational Notes

- The evaluation engine assigns Eligible, Not Eligible, or Needs Manual Review based on evidence completeness, confidence threshold, and rule type.
- Hard disqualification flags are surfaced explicitly rather than hidden inside free-form model output.

6.9 Officer Override and Reason Capture

Preserve officer sovereignty by allowing controlled overrides with structured reasoning and append-only history.

Attribute	Detail
Purpose	Preserve officer sovereignty by allowing controlled overrides with structured reasoning and append-only history.
Prerequisites	Evaluation events exist for the bidder Officer or admin role
Inputs	Selected criterion Final verdict choice Reason code Optional reviewer notes
Outputs	New OVERRIDE event Updated bidder overall status Persistent reason trace

Procedure

- 1 Select the criterion to override in the workbench.
- 2 Choose the final verdict from the dropdown.
- 3 Select an appropriate reason code such as LOW_CONFIDENCE_CONFIRMED or OCR_FALSE_NEGATIVE.
- 4 Add reviewer notes where contextual explanation is useful.
- 5 Click Record Override and verify the confirmation message.

Operational Notes

- Overrides do not mutate prior events. Instead, they create a new event with prior_event_id and a new event hash.

6.10 Audit Timeline Review

Present a grouped, criterion-wise history of automated evaluations and officer overrides for scrutiny, explanation, and export preparation.

Attribute	Detail
Purpose	Present a grouped, criterion-wise history of automated evaluations and officer overrides for scrutiny, explanation, and export preparation.
Prerequisites	At least one evaluation run completed
Inputs	Open Audit action from the bidder console
Outputs	Criterion-grouped event timeline with verdict changes, evidence references, and hash-chain metadata

Procedure

- 1 Open the audit page for the bidder.

- 2 Review the total event count summary.
- 3 Expand each criterion grouping and inspect sequence order, event type, timestamps, verdicts, evidence docs, event hash, and prior event id.
- 4 Use this screen to narrate why a bidder passed, failed, or remained under review.

Operational Notes

- This view is particularly valuable for hackathon judging because it visualizes both transparency and governance discipline.

6.11 Excel and PDF Export

Convert the evaluation ledger into external reporting artifacts that can be shared with stakeholders, auditors, or judges.

Attribute	Detail
Purpose	Convert the evaluation ledger into external reporting artifacts that can be shared with stakeholders, auditors, or judges.
Prerequisites	Audit history available for the target bidder
Inputs	Export Excel or Export PDF action
Outputs	audit_report.xlsx audit_report.pdf

Procedure

- 1 Open the Audit Timeline page.
- 2 Click Export Excel to produce the structured workbook report.
- 3 Click Export PDF to produce the narrative audit report.
- 4 Retrieve files from the browser download flow or the backend audit export directory.

Operational Notes

- Exports are stored under /uploads/audit_exports/// in the backend upload tree.

6.12 Demo Reference Files

Use the repository's provided tender and bidder sample files to demonstrate the complete platform without fabricating evidence.

Attribute	Detail
Purpose	Use the repository's provided tender and bidder sample files to demonstrate the complete platform without fabricating evidence.
Prerequisites	Access to the project workspace
Inputs	Files from Tendor Docs/, Bidder Docs/, and REPO and INFO/

Attribute	Detail
Outputs	Consistent demo narrative backed by real sample artifacts

Procedure

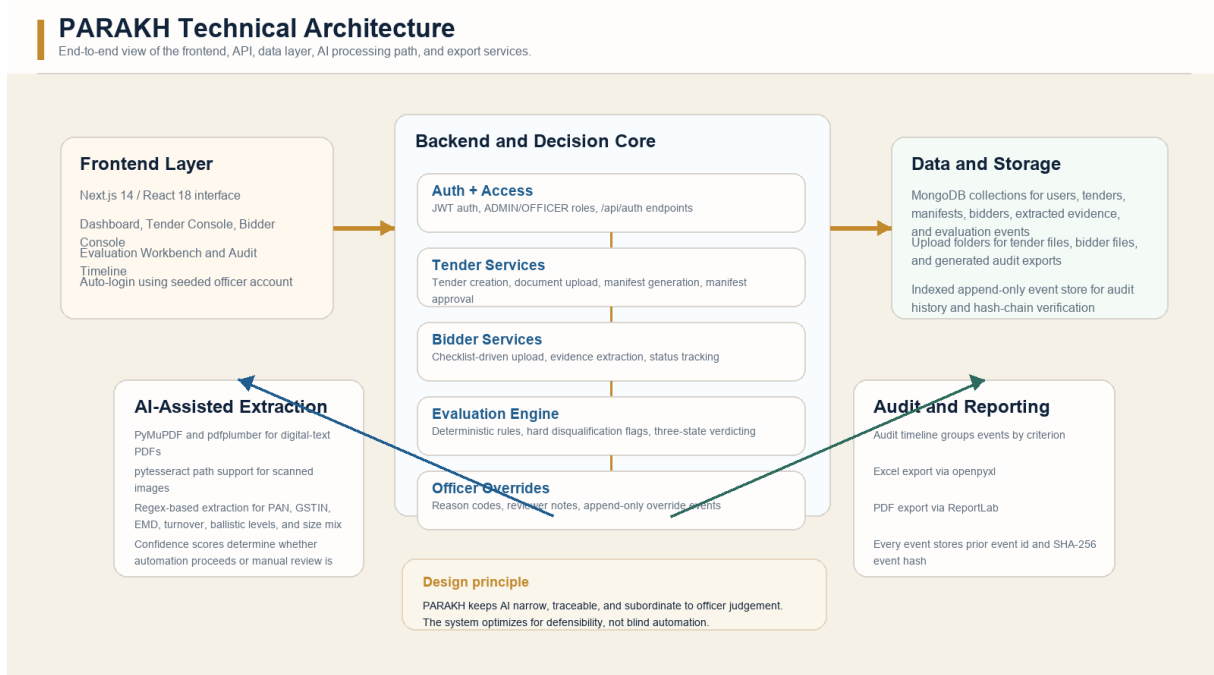
- 1 Use Tendor Docs/ for tender-side uploads such as notice, bid doc, evaluation criteria, and annexures.
- 2 Use Bidder Docs/ for bidder-side checklist uploads.
- 3 Use REPO and INFO/ documents to explain the philosophy, theme fit, and hackathon story during presentation.

Operational Notes

- These reference files are explicitly cited in the repo README as recommended demo inputs.

7. Technical Architecture

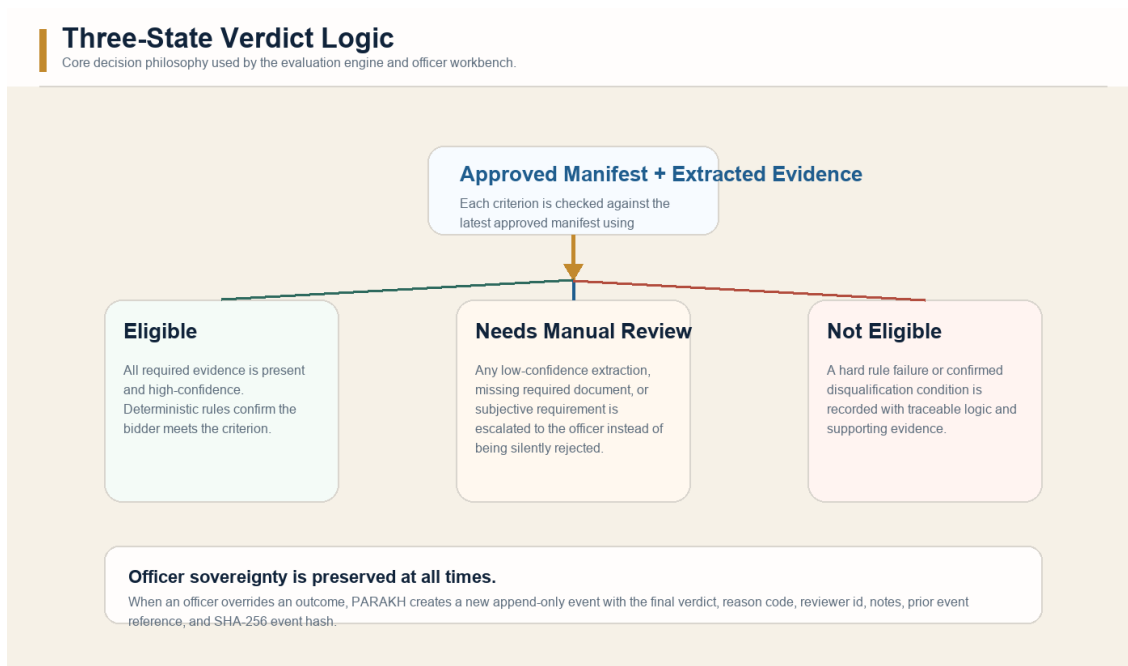
The frontend is a Next.js application that consumes FastAPI endpoints exposed under /api. FastAPI coordinates MongoDB persistence, upload storage, evidence extraction, deterministic evaluation logic, and reporting services. The architecture is intentionally simple enough for a hackathon prototype but already organized around real operational boundaries.



Layer	Implementation detail
Frontend	Next.js 14, React 18, TypeScript, Tailwind CSS
Backend	FastAPI, Pydantic, Motor
Database	MongoDB
Document AI	PyMuPDF, pdfplumber, pytesseract, OpenCV (planned / optional in current environment)

Layer	Implementation detail
Export services	openpyxl, ReportLab
Security	JWT bearer tokens, Role-based access for ADMIN and OFFICER
Infrastructure	Docker Compose, Redis stub for future async workers

Evaluation decision model



8. Data, Files, and Configuration Structure

Operational persistence is split across MongoDB collections, fixture files, and upload folders. This separation makes the prototype easy to reason about during demonstrations and future expansion.

Repository area	Purpose
backend/app/fixtures	Evaluation criteria, disqualification factors, bidder checklist, and policy configuration
backend/app/uploads/tenders	Tender-side uploaded files
backend/app/uploads/bidders	Bidder-side uploaded files
backend/app/uploads/audit_exports	Generated PDF and Excel audit reports
MongoDB users	Stored users and roles
MongoDB tenders	Tender metadata and manifest linkage
MongoDB criteria_manifest_versions	Versioned manifests
MongoDB bidders	Bidder records and checklist state
MongoDB extracted_evidence	Structured evidence extracted from documents
MongoDB evaluation_events	Append-only evaluation and override history

Prototype constraints and known limitations

- Digital-text PDFs work best; OCR quality depends on document clarity and local Tesseract availability.
- The current Redis container is present for future worker integration but is not yet wired into background task execution.
- Clause-classifier and broader LLM-style features are intentionally bounded; final verdicting is rule-based and officer-reviewable.
- The seeded manifest is generated from fixture CSV files rather than full live tender clause extraction.

9. Troubleshooting

The following issues are the most likely to appear during setup or hackathon demonstration. Each entry pairs the symptom with the probable cause and the recommended action.

Frontend opens but data does not load	Backend is not running, NEXT_PUBLIC_API_BASE is incorrect, or the login token is stale.	Confirm backend health at /health, verify NEXT_PUBLIC_API_BASE=http://localhost:8000/api, and reload the page.
Bidder ingestion returns weak or empty evidence	The file is image-heavy, OCR tooling is unavailable, or the source file is low quality.	Use clearer sample files, ensure Tesseract is installed for scanned inputs, and prefer digital PDFs when possible.
Manifest cannot be edited	The current version is already approved.	Generate a new draft manifest or edit only while the manifest status is DRAFT.
Evaluation remains under review	Required evidence is missing, low confidence, or the criterion is subjective by design.	Open the workbench, inspect missing docs or low-confidence fields, and use an officer override if justified.
Exports do not appear	No audit history exists yet or export directory permissions are blocked.	Run evaluation first, then re-open the audit page and repeat export after confirming backend write access.
MongoDB connection error	MongoDB is not running or MONGO_URI is incorrect.	Start MongoDB first, then confirm the configured connection string matches the active runtime.
Frontend dev server fails on Windows PowerShell	PowerShell script policy blocks the npm shim.	Use cmd /c npm.cmd install and cmd /c npm.cmd run dev as documented in the README.

10. Operational Best Practices

- Approve a manifest only after thresholds, descriptions, and guidance notes reflect the tender being demonstrated.
- Use higher-quality digital PDFs whenever possible; reserve scanned image demos for cases where OCR behavior itself is part of the story.
- Treat Needs Manual Review as a feature, not a weakness. It shows that the platform refuses false certainty when evidence is incomplete or subjective.

- Always finish a live demo on the audit timeline or exported report so judges see the governance advantage clearly.
- For non-OEM bidders, remember to demonstrate how missing OEM Authorization affects disqualification logic.

11. Glossary

Term	Definition
Tender	A government request inviting suppliers to bid for goods or services.
Bidder	A supplier or consortium participating in the tender.
Manifest	The structured evaluation blueprint generated from tender-side criteria.
Criterion	One rule or requirement to test during bidder evaluation.
Evidence	The extracted text and structured fields sourced from bidder documents.
Hard Disqualification	A confirmed failure condition that should lead to rejection unless policy allows otherwise.
Graduated Criterion	A criterion that may need officer interpretation instead of full automation.
Needs Manual Review	A verdict state used when confidence is low, evidence is missing, or the rule is subjective.
Reason Code	A structured label used to explain an officer override.
Event Hash	A SHA-256 digest stored for each evaluation or override event to make the audit trail tamper-evident.
RTI	Right to Information; a transparency framework under which procurement decisions may later be questioned.
CAG / Vigilance	Oversight and audit mechanisms that can review procurement decisions and supporting records.

12. Appendices

12.1 Seeded evaluation criteria

Criterion	Why it matters
EMD Validity	Commercial seriousness and bid security.
Legal Standing	Baseline authorization and tax identity.
Protection Level	Life-safety compliance against ballistic standards.
Design/Pattern	Tender-specific technical conformity that often needs officer review.
Sizing Accuracy	Operational fit against the required size distribution.
Turnover Depth	Financial capacity to execute the contract.
Supply History	Experience and comparability of prior work.

12.2 Bidder checklist reference

Document	Operational purpose
Industrial License	Validate manufacturing authorization for bullet resistant jackets.
EMD Proof	Confirm earnest money deposit amount, instrument type, and validity.
Ballistic Test Reports	Verify BIS Level-6 protection for front, back, and side panels.
Technical Compliance	Confirm camouflage pattern alignment and required size mix.
Annual Turnover Certificate	Measure bidder and OEM average turnover against thresholds.
Past Performance Records	Assess large-order supply history for comparable items.
OEM Authorization	Validate supply-chain legitimacy for non-OEM bidders.
PAN and TIN Certificates	Establish tax identity and legal standing.
Proof of Experience	Confirm minimum industry tenure.
Additional Docs (ATC)	Capture extra administrative documents requested by tender conditions.

12.3 Override reason codes

Reason code	When to use it
LOW_CONFIDENCE_CONFIRMED	The officer checked low-confidence evidence and still accepts or rejects the extraction outcome.
ADDITIONAL_EVIDENCE_ACCEPTED	The decision was clarified by supplementary evidence or context.
POLICY_EXCEPTION_APPROVED	A policy-authorized exception was applied under officer responsibility.
OCR_FALSE_NEGATIVE	OCR missed a valid signal that the officer confirmed manually.
OFFICER_INTERPRETATION	Human interpretation was necessary because the criterion is subjective or contextual.
DOCUMENT_SCOPE_CLARIFIED	The officer determined that the document does or does not satisfy the intended scope.

12.4 Core commands for presenters and developers

Scenario	Command or script
Docker startup	<code>docker-compose up --build</code>
Windows local MongoDB	<code>.\start-mongo-local.cmd</code>
Windows local backend	<code>.\start-backend-local.cmd</code>
Windows local frontend	<code>.\start-frontend-local.cmd</code>
Backend dev	<code>uvicorn app.main:app --reload</code>
Frontend dev	<code>npm run dev</code>

End of manual. This document is intended to function as both an operational runbook and a hackathon presentation aid for PARAKH.